

PHENIKAA UNIVERSITY
PHENIKAA SCHOOL OF COMPUTING



COURSE: SOFTWARE ARCHITECTURE
HOTEL MANAGEMENT SYSTEM

INSTRUCTOR: Vũ Quang Dũng
GROUP 7: Phạm Thế Thủy Hoàng (23010960 – K17-
KTPM)
CREDIT CLASS: CSE703110-1-2-25(N01)

Ha Noi, March 6, 2026

Contents

1. Executive Summary	3
2. Testing & Verification	3
2.1 Unit Testing	3
2.2 Integration Testing.....	5
2.3 Verification Proof: Booking – Payment – Check-in Flow	7
3. Conclusion & Reflection.....	11
3.1 Lessons Learned	11
3.2 Future Improvements.....	11

1. Executive Summary

This project implements a Hotel Booking Management System that supports the end-to-end workflow of a real hotel: customers can browse rooms, create bookings, make payments, manage their wallet, and view booking history; receptionists can check guests in and out and manage service requests; administrators can manage rooms, services, staff, coupons, and reports. The system is architected using a layered architecture on top of Flask, with clear separation between controllers (HTTP layer), services (business logic), repositories (data access), and models (domain entities).

The backend lives under SRC/Arch and exposes REST-style JSON APIs (e.g. /api/bookings, /api/rooms, /api/auth/login, /api/wallet) through controller classes such as booking_controller.py, room_controller.py, user_controller.py, payment_controller.py, coupon_controller.py, checkin_controller.py, staff_controller.py, report_controller.py, and wallet_controller.py. These controllers rely on service classes (e.g. booking_service.py, room_service.py, payment_service.py, coupon_service.py) that encapsulate business logic and call repository classes (e.g. booking_repository.py, room_repository.py, coupon_repository.py) to access a MySQL database.

The frontend is implemented in SRC/UI as a set of HTML/CSS/JS pages for different roles: public pages for browsing rooms, authentication pages for login and registration, user pages for dashboard, my bookings, payment, and wallet, and admin pages for room and staff management and reports. The final outcome is a working multi-role hotel management application where the architecture supports modifiability, maintainability, and basic security and scalability for small-to-medium usage.

2. Testing & Verification

The main goal of testing is to ensure that:

- Booking and room availability logic works correctly (no overbooking).
- Payments and wallet operations are consistent with booking states.
- Authentication and authorization are enforced for protected endpoints.
- The frontend can interact smoothly with the backend APIs and present correct data.

2.1 Unit Testing

Unit tests verify individual components (functions, methods, or classes) in isolation, usually with dependencies mocked or stubbed. In this project, unit testing focuses on the business and data-access layers so that validation rules, state transitions, and data mapping behave as specified without requiring a live database or HTTP server.

- Service layer: `booking_service.py`, `coupon_service.py`, `payment_service.py`, `wallet_service.py`, and other services are tested for core logic—e.g. date overlap validation, coupon validity and discount calculation, wallet balance checks before payment, and refund amount computation on cancellation. Dependencies such as repositories are mocked so that tests are fast and deterministic.
- Repository layer: Repository methods (e.g. `booking_repository.py/BookingRepository.find_available_rooms_by_date_range`, `WalletRepository.update_balance`) can be unit-tested with an in-memory SQLite database or with mocks of the database connection to assert correct SQL behaviour and mapping from rows to domain models.
- Validation and helpers: Input validation (date ranges, positive amounts, required fields) and utility functions (e.g. in `utils` or `middleware`) are covered by unit tests to prevent invalid data from propagating to the database or API responses.

Tools such as `pytest` or `unittest` can be used, with `pytest` fixtures for shared setup and mocks (e.g. `unittest.mock` or `pytest-mock`) for repository and external dependencies.

Example unit tests:

1. BookingService – reject past check-in dates

```
def test_create_booking_rejects_past_check_in_date():
    service = BookingService()
    past_date = "2020-01-01"
    with pytest.raises(ValueError):
        service.create_booking(
            guest_name="Alice",
            room_type="standard",
            check_in_date=past_date,
            total_price=100.0,
        )
```

2. BookingRepository – mapping database row to Booking model

```
def test_row_to_booking_maps_row_correctly():
    repo = BookingRepository()
    row = {
        "id": 42,
        "guest_name": "Bob",
        "room_type": "deluxe",
        "check_in_date": date(2026, 2, 10),
        "check_out_date": date(2026, 2, 12),
        "total_price": 300.0,
        "status": "pending",
        "room_id": 7,
```

```

        "user_id": 5,
    }
    booking = repo._row_to_booking(row)
    assert booking.id == "42"
    assert booking.room_id == 7
    assert booking.user_id == 5

```

3. PaymentService – prevent double payment

```

def test_process_payment_rejects_when_fully_paid(monkeypatch):
    service = PaymentService()
    # Fake repositories to avoid real DB
    monkeypatch.setattr(service, "booking_repo", FakeBookingRepo(total_price=100.0))
    monkeypatch.setattr(service, "repo", FakePaymentRepo(existing_amount=100.0))
    with pytest.raises(ValueError):
        service.process_payment(booking_id=1, amount=50.0, payment_method="cash")

```

2.2 Integration Testing

Integration tests verify that multiple components work together correctly—for example, API endpoints with the real (or test) database, or controller → service → repository flows. They ensure that the contracts between layers are respected and that end-to-end behaviour within the backend is consistent.

- API + database: Key flows are tested by sending HTTP requests to the Flask app (e.g. using Flask test client) against a dedicated MySQL database or an in-memory SQLite instance. Examples include: POST /api/bookings creates a booking and updates room availability; POST /api/payments correctly deducts wallet balance and updates booking status; GET /api/rooms returns only available rooms for the given date range.
- Authentication and authorization: Endpoints protected by `require_auth`, `require_admin`, or `require_receptionist_or_admin` are tested with valid and invalid X-User-Id headers (and optionally role) to confirm that forbidden access returns 401/403 and that allowed roles succeed.
- Cross-layer flows: Integration tests cover scenarios such as create booking → process payment → check-in → check-out, and cancel booking → refund to wallet, ensuring that service and repository interactions and transaction boundaries behave correctly under a single application context.

These tests may use the same Flask application factory and a test configuration that points to a test database, with cleanup (e.g. transaction rollback or truncated tables) between tests to avoid side effects.

Example integration tests:

1. Booking API + room availability

```
def test_create_booking_reserves_room(client, db):
    # Arrange: seed one available room of type "standard" in the test DB
    room_id = seed_room(db, room_type="standard", status="available")
    payload = {
        "guest_name": "Alice",
        "room_type": "standard",
        "check_in_date": "2026-02-10",
        "check_out_date": "2026-02-12"
    }
    # Act: call POST /api/bookings via Flask test client
    response = client.post("/api/bookings", json=payload, headers={"X-User-Id": "1"})
    # Assert: booking is created and room status becomes 'reserved'
    assert response.status_code == 200
    data = response.get_json()
    assert data["booking"]["room_id"] == room_id
    assert get_room(db, room_id).status == "reserved"
```

2. Payment API + wallet deduction

```
def test_wallet_payment_deducts_balance(client, db, wallet_factory):
    wallet = wallet_factory(user_id="1", balance=500.0)
    booking_id = seed_booking(db, user_id="1", total_price=200.0)
    payload = {
        "booking_id": booking_id,
        "amount": 200.0,
        "payment_method": "cash"
    }
    response = client.post("/api/payments", json=payload, headers={"X-User-Id": "1"})
    assert response.status_code == 200
    updated_wallet = get_wallet(db, "1")
    assert updated_wallet.balance == wallet.balance - 200.0
```

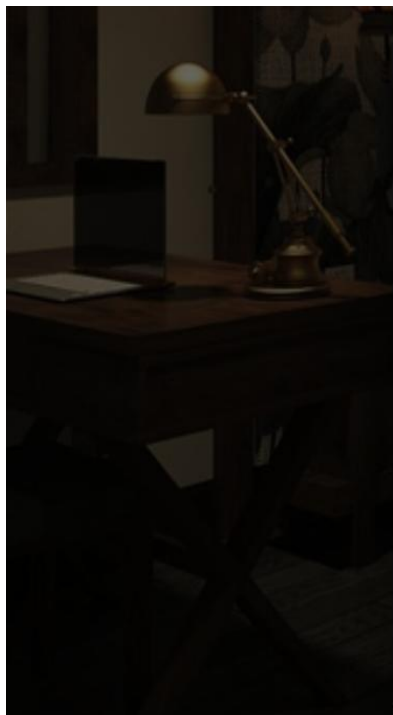
3. Authorization on admin endpoint

```
def test_normal_user_cannot_access_admin_rooms(client, db):
    # User without admin role tries to access admin-only rooms endpoint
    response = client.post("/api/rooms", json={}, headers={"X-User-Id": "normal-user"})
    # Expect 403 Forbidden returned by require_admin middleware
    assert response.status_code == 403
```

2.3 Verification Proof: Booking – Payment – Check-in Flow

Test Scenario: A user successfully books a room, pays using wallet, and is checked in and later checked out by a receptionist.

Step	Action	Expected Result	Actual Result	Status
1	User registers and logs in via UI	User account is created and authenticated	OK	Pass
2	User browses rooms and selects room type and date range	Available rooms are displayed	OK	Pass
3	User confirms booking (POST /api/bookings)	Booking is created with pending status and assigned room_id	OK	Pass
4	User opens payment page and pays via wallet	Wallet balance is verified, payment is processed, booking status updated to paid	OK	Pass
5	Receptionist views upcoming arrivals and performs check-in	Check-in record is created, room status is updated	OK	Pass
6	Receptionist performs check-out at end of stay	Check-out summary is generated, room is released	OK	Pass



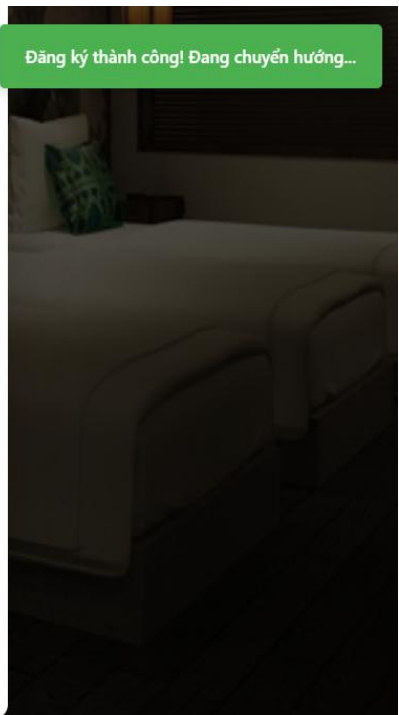
Full Name *

Phone Number *

Password *

Password must be at least 6 characters
Confirm Password *

Register
[Already have an account? Login now](#)
[— Back to home](#)



Đăng ký thành công! Đang chuyển hướng...

Figure 13: Register completed

◆ Demacia

HOME

MY BOOKINGS

ROOMS

CONTACT

WALLET

23010283@st.phenikaa-uni.edu.vn

Logout

VI

EN

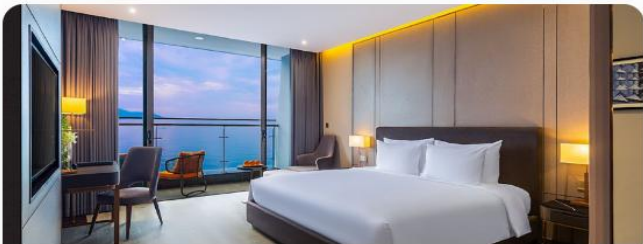
Explore Rooms

Complete list of room types, prices and availability for your stay.

Room List

View all available rooms and their current status.

Search room number, type, status...

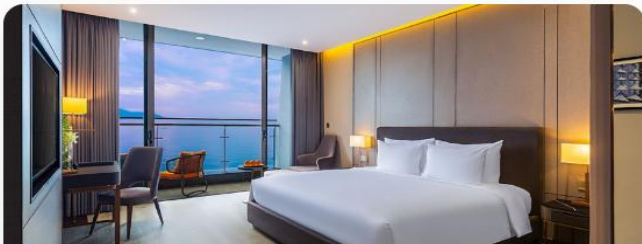


Room 100 · Senior

2 guests Free WiFi Available

500,000 VND/night

→



Room 101 · Senior

2 guests Free WiFi Available

500,000 VND/night

→

Figure 14: Search rooms

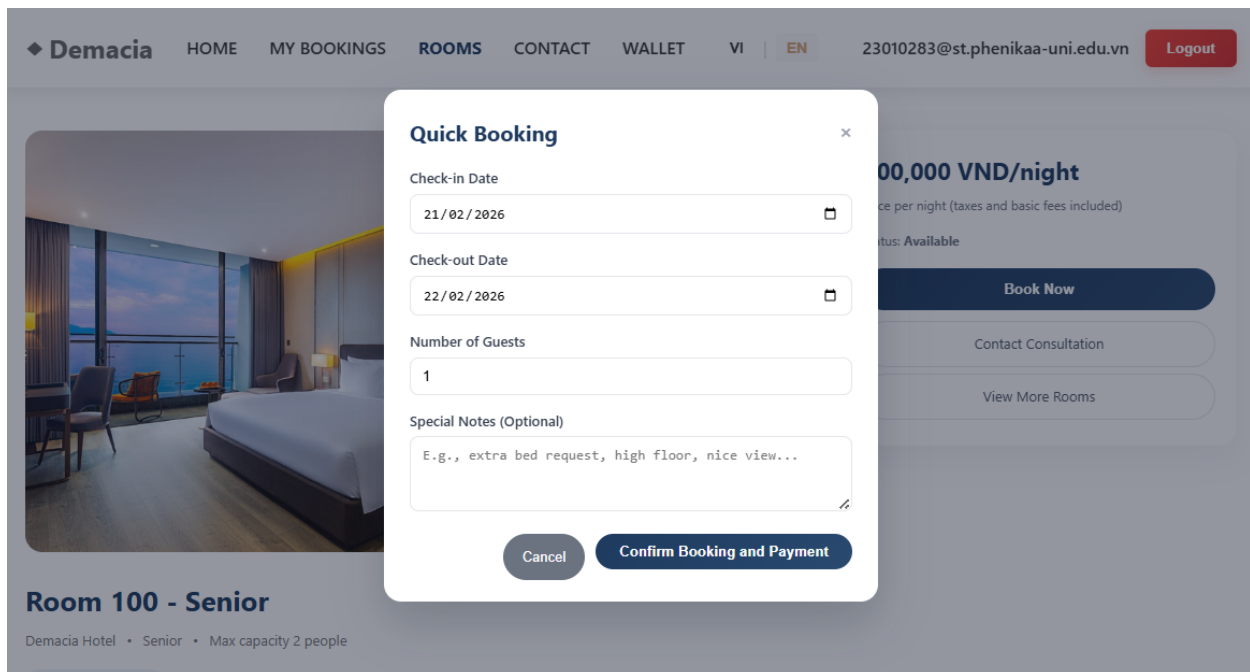


Figure 15: Booking

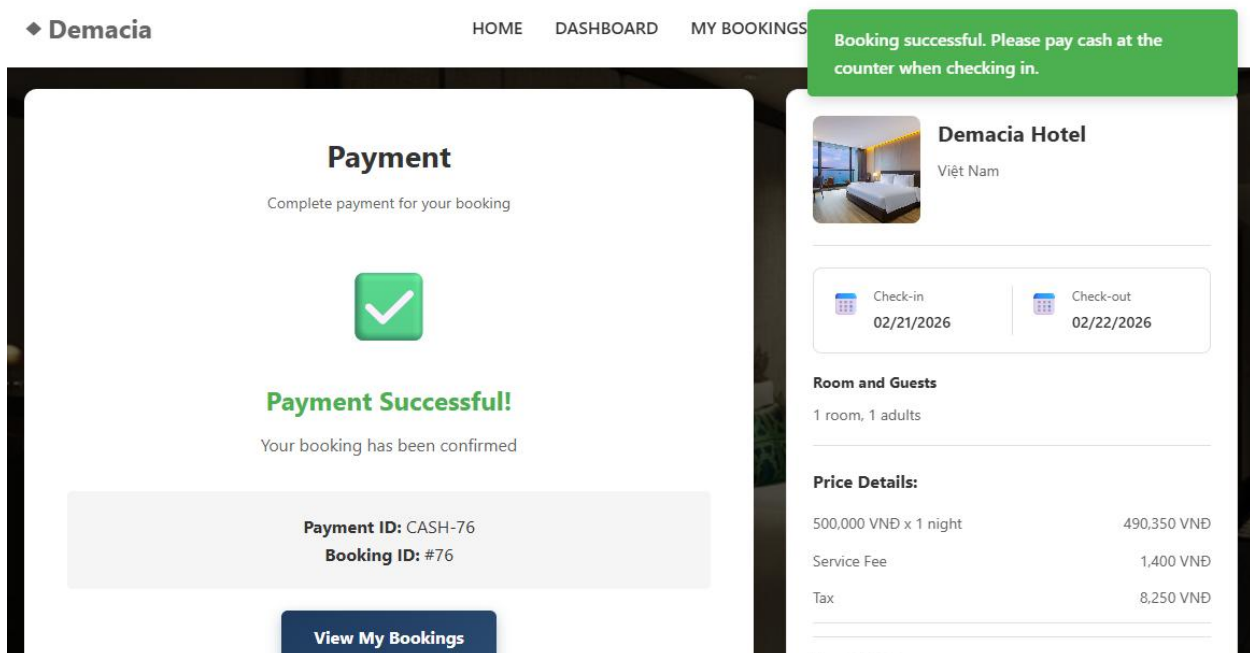


Figure 16: Payment completed

◆ Demacia

Receptionist Panel

CONTACT

reception

Logout

VI

EN

Manage Bookings

Check-in

Check-out

Manage Service Requests

Payment History

Manage Bookings

All

ID	Guest Name	Room Type	Check-in Date	Check-out Date	Total Amount	Status	Actions
#76	An Tran Ngoc	Senior	21/2/2026	22/2/2026	500.000 VNĐ	Pending Confirmation	Confirm Cancel Booking
#75	1	Senior	30/1/2026	31/1/2026	500.000 VNĐ	Checked Out	
#74	2	Senior	30/1/2026	31/1/2026	100.000 VNĐ	Checked Out	
#73	1	Senior	30/1/2026	31/1/2026	500.000 VNĐ	Checked Out	
#72	4	Senior	30/1/2026	31/1/2026	500.000 VNĐ	Cancelled	

Figure 17: Manage bookings

◆ Demacia

Receptionist Panel

CONTACT

reception

Logout

VI

EN

Check-out for booking #76 - An Tran Ngoc

Room Type: Senior

Check-in Date: 21/2/2026

Check-out Date: 22/2/2026

Payment Details:

Room Price: 500.000 VNĐ

Additional Service Costs: 0 VNĐ

Total: 500.000 VNĐ

No additional services

Back

Process Check-out

Figure 18: Checkout

Additional tests include:

- Attempting to book the same room and date range twice, which should fail due to the SQL overlap logic in `BookingRepository.find_available_rooms_by_date_range`.
- Attempting to access admin endpoints as a normal user, which should be blocked by `require_admin` or `require_receptionist_or_admin` middleware.
- Applying expired or inactive coupons, which should be rejected by `CouponService`.
- Verifying that cancellation triggers appropriate room release and, if applicable, wallet refunds, and that booking status is updated to 'cancelled'.

3. Conclusion & Reflection

This project successfully implements a multi-role Hotel Booking Management System using a layered Flask backend and a static HTML/CSS/JavaScript frontend. The layered structure with controllers, services, repositories, and models has proven effective in separating concerns, making it easier to reason about booking, payment, and reporting flows. The system satisfies the main functional requirements (room browsing, booking, payment, check-in/check-out, wallet, coupons, services, reporting) and achieves key architectural goals such as modifiability, maintainability, and basic security.

3.1 Lessons Learned

Value of Clear Layering: Explicit separation between controllers, services, and repositories significantly reduced coupling. When business rules changed (e.g., new validation for booking dates or coupon constraints), modifications could be localized mainly in the service layer.

Handling Data Consistency: Implementing correct booking and payment flows required careful handling of transactions and state transitions (pending, paid, cancelled, checked_in, checked_out). The repository methods and service logic had to be designed to avoid partial updates (e.g., reserving a room but failing to create a booking).

Security & Roles: Introducing middleware-based decorators for authentication and role-based access control made it easier to secure endpoints consistently across the application. This pattern helped enforce that only appropriate roles can perform sensitive operations like managing rooms, staff, or viewing reports.

Integration with a Real Database: Working with MySQL and explicit SQL queries (instead of an ORM) provided deeper understanding of SQL, transactions, and how to map between relational tables and Python objects.

3.2 Future Improvements

1. Strengthen Authentication & Authorization:

- Adopt a more robust token-based authentication mechanism (e.g., JWT) with secure storage on the client and token refresh logic.
- Enhance password policies, add account lockout after repeated failed logins, and introduce audit logging for security-sensitive operations.

2. Enhance Reporting:

- Extend ReportService and related repositories to produce richer analytics such as revenue by room type, monthly/seasonal trends, average length of stay, and service usage statistics.
- Provide export options (CSV/PDF) and more visual dashboards in the admin UI.

3. Improve Scalability & Deployment:

- Containerize the application (Docker), use environment-specific configurations, and deploy on a cloud platform.
- Introduce a reverse proxy/load balancer and, if necessary, separate the booking, payment, and reporting domains into independently scalable services.

4. UX & UI Enhancements:

- Improve responsiveness and interactivity of the frontend, potentially integrating a modern frontend framework for complex admin views.
- Add user notifications (email or in-app) for booking confirmation, payment receipts, and check-in reminders.

5. Logging, Monitoring & Error Handling:

- Implement structured logging and monitoring (e.g., using a logging framework and a dashboard) to quickly detect issues in production.
- Standardize error responses across controllers to provide consistent feedback to Frontend developers and end users.